

Whole-Protocol Admission for Restricted Python on RT Hardware

Anonymous Author(s)

Abstract

Ray-tracing APIs expose fast traversal through callback protocols spanning roles, semantic and physical layouts, continuation, and executable identity. Existing systems offer strong local checks, Python access, and pipeline construction. The primary materials we examined, however, do not state the same joint admission condition over source, generated artifacts, loaded executable, and result publication.

We present RTDL, a bounded protocol-carrying compilation and admission design for Python-authored RT computations. It jointly checks five route seams: role/effect closure, semantic ownership, physical binding, fail-closed continuation, and executable identity. Typed effects reach hardware control only through trusted topology-specific wrappers. The implementation has two stable public constructors and a bounded corpus spanning triangle, custom-AABB, curve, and sphere leaves; it is not arbitrary Python, semantic inference, or generic topology lowering. A sealed-snapshot exam selected a sphere any-hit count and produced two OptiX launches with 12/12 oracle checks. A checker that does not import the compiler validates five generated-code property classes in 20 registered instances, but only over finite target structures.

Across two NVIDIA GPU generations, we evaluate 160 registered cells containing 20,480 timed samples. At the prepared endpoint, public ahead-of-time RTDL median latency is $1.077\text{--}1.175\times$ a direct CUDA/OptiX implementation. First-result measurements are adverse: post-import RTDL is $1.560\text{--}1.837\times$ a strong PyOptiX device-continuation baseline by median, with a $2.377\times$ worst block. Thus the contribution is bounded protocol admission with near-direct prepared latency on two tasks, not intrinsic speedup, general lowering, demonstrated usability, or a soundness theorem.

CCS Concepts: • Software and its engineering → Compilers.

Keywords: ray tracing, restricted Python, protocol admission, GPU compilation

ACM Reference Format:

Anonymous Author(s). 2027. Whole-Protocol Admission for Restricted Python on RT Hardware. In *Proceedings of IEEE/ACM International Symposium on Code Generation and Optimization (CGO '27)*. ACM, New York, NY, USA, 8 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Modern ray-tracing systems expose programmable stages around fixed-function traversal. OptiX, Vulkan ray tracing, and DirectX Raytracing let programs choose geometry representations, payload conventions, hit behavior, and launch topology [11, 13, 14, 17]. This flexibility has enabled non-rendering workloads to use RT cores, but it also creates an unusual language boundary. A callback can be locally well typed and still be unsafe or semantically wrong because it is paired with the wrong build-input kind, stale generated code, an incompatible payload, or an invalid lifecycle transition.

PyOptiX and SlangPy expose RT to Python; OWL and Luisa automate substantial pipeline construction; DXR payload qualifiers and Slang capabilities enforce stage and target constraints [9, 13, 15, 16, 19, 22]. These are strong mechanisms. We instead ask how a compiler should gate one relation spanning locally checked callbacks, same-width application meanings, physical artifacts, device continuation, and the exact executable. The checked primary materials do not document that joint condition; in several cases the answer is unknown, not proven absent:

Can restricted Python admit a bounded RT route by jointly checking semantic, physical, continuation, and identity facts while retaining near-direct prepared latency?

For a narrow domain, RTDL maps typed restricted callbacks to compiler-owned, topology-specific lowerers. It binds source, PTX, ABI, and native symbols; gates lifecycle transitions and output publication; and provides an independent offline checker for selected generated-code properties. Its canonical plan is non-executable, while concrete lowering remains trusted and topology-specific.

This paper makes one core systems contribution, an implementation contribution, and a bounded evidence contribution:

1. A *protocol-carrying admitted executable*: one decision over five cross-artifact relations gates materialization, native load, and publication.
2. A restricted-Python implementation in which typed callback effects are flattened into an ABI and consumed by trusted topology-specific OptiX wrappers. It has two stable public constructors and four leaf kinds in its bounded corpus; the wrappers remain in the trusted computing base (TCB).
3. Finite mutation, sealed-composition, target checking, and two-generation performance evidence. Prepared

Table 1. Whole-protocol obligations in the current system.

Obligation	Admission question
Role/effect	Is each operation legal in this callback role?
Semantic ABI	Do payload and result meanings agree end to end?
Physical ABI	Do widths, alignments, slots, and row layouts agree?
Topology	Does the callback set match the build-input and leaf kind?
Identity/state	Is the checked executable the loaded executable, and is every state transition legal?

public latency is $1.077\text{--}1.175\times$ Direct; adverse first-result, receipt, generality, and authoring results delimit the contribution.

2 The Protocol Mismatch

2.1 Why callback typing is insufficient

A ray callback runs inside a protocol distributed across host and device code. Its legal effects depend on its role: an any-hit callback may ignore or terminate an intersection, whereas a miss callback cannot. Its payload fields must agree with every producer and consumer. Its program group must match the acceleration-structure leaf kind. The launch must occur only after module, pipeline, shader-binding-table, geometry, and output state have been prepared. Finally, the code checked by the compiler must be the code loaded by the runtime.

These are related to typestate, interface automata, and session-style protocol reasoning [2, 7, 21], but the FFI and generated-code boundary adds an identity problem familiar from linking and foreign interfaces [4, 18]. Table 1 summarizes the obligations used here.

2.2 A concrete semantic-ABI failure

Consider two acceleration-structure primitives at physical positions 0 and 1 whose application identifiers are 10 and 20. Queries 100 and 101 should emit the pairs (100, 10) and (101, 20). If an intersection producer instead writes the physical position into attribute slot zero while the consumer interprets that slot as an application identifier, the apparent rows are (100, 0) and (101, 1). Every value remains a valid u32; width, slot, and row layout need not expose the semantic substitution. This is an illustrative witness, not a retained execution of the defective route.

The defect is not a machine-type error but a disagreement over what the bits mean. In RTDL, a trusted bounded-relation schema declares that attribute slot zero denotes an application identifier; the compiled relation contract carries that row source into a separately derived target

projection. Admission compares the two compiler representations. Existing tests mutate only this fact and obtain CP002_ATTRIBUTE_ABI_OWNERSHIP_MISMATCH; an integrated projection mutation is rejected before native load. This proves check liveness, not automatic inference of application meaning or end-to-end execution of the illustrative defect. Both derivations remain in one compiler TCB, and a wrong but internally coherent trusted schema can pass.

2.3 Failure model

We distinguish four outcomes. *Reject* means admission fails before a launch. *Status failure* means execution reports failure and output is not published. *Wrong output* means a worker reports success but an independent oracle disagrees. *Unverified output* means a value exists without the required receipt or oracle evidence. The distinction prevents a status gate from being described as numerical correctness and prevents a generated plan from being described as an executable lowering.

A focused mock reproduced an unrepaired provider double fault: secondary bind or close failure can replace the primary exception and lose cleanup retry ownership. It released no output and did not occur in retained successful GPU workers. A native fork bypassing the registered Python at-fork hook can evade the cached-PID guard; inherited owners after such forks are unsupported.

3 Bounded Language and Admission

3.1 Callback representation

The source language is restricted Python, not arbitrary Python. Source is parsed through an AST allowlist and is not imported or executed for discovery. The front end resolves declared immutable records, frozen constants, same-unit acyclic helpers, and selected intrinsics, then produces canonical typed IR. Deserialization reconstructs typed objects and reruns verification; serialized IR is not itself an authority. Table 2 summarizes the representation.

The current resource contract admits at most 32 payload slots and eight attribute slots, trace depth one, callable depth zero, bounded helper depth, and statically bounded iteration. The numeric policy requires explicit binary32 behavior and fail-closed handling of prohibited integer overflow and nonfinite effects. These are language and resource checks; they do not prove that a chosen ray encoding computes the intended application function.

3.2 Role-indexed effects

Seven language roles participate in an admitted unit. bounds runs during preparation; make_ray and finalize are language leaves invoked by a trusted ray-generation wrapper; and intersection, any_hit, closest_hit, and miss map to target-stage behavior. User code cannot invoke raw traversal. Instead, each role returns one typed effect from a

Table 2. Implemented bounded representation. Rejected forms have no fallback lowering.

Form	Accepted subset	Rejected boundary
Types	Booleans; fixed-width integers and floats; 2–4-lane vectors; tuples; acyclic immutable records; checked read-only views	Raw pointers, mutable containers, opaque objects, and recursive records
Expressions	Typed names and literals, record fields, checked indexing, arithmetic, comparisons, bit operations, constructors, selected pure intrinsics, and bounded helper calls	Reflection, dynamic dispatch, allocation, foreign calls, and unknown attributes
Statements	Typed binding, loop-local type-preserving update, two-sided conditionals, statically bounded loops, role-effect return, and helper-value return	Dynamic loops, recursion, exceptions, generators, async control, and not-all-paths-return
Manifest	Payload/output records, machine attributes, constants, numeric policy, resource budget, topology, continuation, and linkage	Self-minted physical authority, unreviewed production linkage, and unsupported budgets
Program	Canonical source and IR identities, records, functions, manifest, and separately derived effect digest	Unknown fields, malformed schema, stale identities, or unverified deserialization

role-specific set: bounds returns an AABB, ray construction returns a trace request, intersection returns hit or no-hit, hit callbacks return payload plus continue/ignore/terminate control, miss returns payload, and finalize returns an output.

Topology makes these requirements relational. Custom AABBs require bounds, ray construction, intersection, miss, finalize, and at least one hit role. Built-in triangles forbid bounds and intersection because hardware owns them, but retain ray construction, miss, finalize, and at least one hit role. An any-hit role also requires an explicit delivery and continuation contract; the presence of that declaration does not prove order independence for arbitrary callback logic.

The executable path makes the authority split explicit. Verification accepts a role-indexed typed effect and checks its exact fields. ABI lowering flattens the effect and assigns effect/status tags; a generated Numba device leaf writes those fields and tags. A trusted topology wrapper invokes the leaf, checks its tag and first-error state, and only then performs operations such as payload updates, `optixIgnoreIntersection()`, or fail-closed termination. Thus untrusted callback code does not directly issue traversal control, but the specialized wrapper remains trusted.

3.3 Fail-closed judgments

Expression checking derives $\Gamma \vdash e : \tau$. Statement checking also returns the output environment, all-paths-return bit, conservative static work, and observed effects:

$$\Gamma; r \vdash s \Rightarrow \langle \Gamma', q, n, \epsilon \rangle. \quad (1)$$

For each role r , verification enforces its exact signature and

$$M \vdash F_r : \epsilon \quad \text{with} \quad \epsilon \subseteq \text{AllowedEffects}(r). \quad (2)$$

Names are single-assignment except for type-preserving updates inside static loops; loop locals cannot escape; both non-returning branches define the same typed names; and

every role/helper path returns. Whole-unit checking validates schema/source identity, acyclic records and helpers, numeric and resource budgets, role cardinality, topology, linkage, and required external physical authority:

$$\mathcal{G}; \mathcal{R} \vdash Q \Downarrow \text{Verified}(h_{\text{IR}}, h_{\text{effects}}). \quad (3)$$

Here $\mathcal{G}$ is an out-of-program geometry authority and $\mathcal{R}$ is the compiler-owned role topology. The judgment is explanatory notation for implemented checks, not a mechanized soundness theorem.

Admission is conjunctive. A route is accepted only if its schema is valid, its roles permit all effects, its semantic and physical ABIs agree, a trusted lowerer exists for its topology, the generated executable identity is bound, and the runtime lifecycle can reach the prepared state. Failure of any one predicate rejects the route; there is no “best effort” projection into a nearby executable.

3.4 Layered whole-protocol admission

Table 3 identifies where each obligation originates. This separation matters because target evidence is regenerated within one trusted implementation; it is not an independent proof of compiler correctness.

3.5 Plan versus executable lowering

The canonical planner normalizes the admitted schema and emits a declarative plan. That plan is intentionally marked non-executable. It does not synthesize arbitrary CUDA or OptiX callbacks. Instead, an accepted route names a compiler-owned, topology-specific lowerer and wrapper. The lowerer emits source and PTX, binds native symbols, constructs the program groups and shader-binding table, and records their identities. This design supplies a common admission surface while keeping concrete backend knowledge visible in the TCB.

Table 3. Authorities compared by whole-protocol admission.

Obligation	Declaration authority	Target/runtime evidence	Exact limit
Role/effect	Verified role topology and returned effects	Reverified compiled callback ABI variants	Common compiler TCB; no arbitrary effect inference
Semantic ABI	Trusted family schema and fixed-route nominal row-source declaration plus machine types	Compiled relation contract and callback ABI	Same compiler TCB; meaning is declared, not inferred; generic u32 has no nominal meaning
Physical ABI	Typed plan for geometry, buffers, hit channel, result, and reducer	Rebuilt target facts and bound storage	Exact-plan admission, not arbitrary physical synthesis
Continuation	Completeness/capacity policy and required status-before-use order	Status vocabulary, compiled continuation, runtime status gate	Bounded supported modes; not a confluence theorem
Identity/lifecycle	Materializer-selected source/PTX/provider identity and state machine	Loaded bytes, prepared objects, execution status, output digest, and optional receipt	Hash equality is identity coherence, not semantic correctness

Shared schema and lifecycle machinery do not make device code generic. The prospective sphere route in Section 5.2 required about 2,635 lines of topology-specific code and 28 modified compiler lines. It shows that a trusted route can reuse three sealed shared files, not automatic topology lowering.

3.6 Executable identity and lifecycle

The system binds hashes and shape metadata for generated source, PTX, ABI, native provider symbols, and the public route. Runtime preparation checks the binding before execution. The admitted lifecycle is:

static verify → materialize/bind → prepare → execute/status gate → output → receipt.

The public path synchronously checks native and compact status and any supplied expected output before return; the experiment then checks value and digest against its worker oracle. It does not rehash disk files per launch. The preregistered per-execution receipt requirement is unmet: 4,096 timed A executions have only 32 separate post-loop diagnostic executions, one detailed receipt per A worker. We claim status/optional-output gating and experimental validation, not per-execution receipt coverage.

4 Implementation Scope

The implementation uses Python for the public authoring surface and C++/CUDA with OptiX for trusted lowering and execution. This is a compiler/runtime prototype rather than a pure-Python implementation. Table 4 separates stable public constructors from the broader bounded route corpus.

The route corpus is broader than the stable facade but narrower than OptiX. Unsupported geometry options, motion modes, callback combinations, and payload shapes remain outside the contract. Extension requires adding or auditing a trusted lowerer; a matching schema alone is insufficient.

Table 4. Implemented scope; presence is not complete feature coverage.

Surface	Current evidence
Stable facade	Two constructors: custom-AABB bounded canonical relation; built-in triangle checked u64 reduction
Stable kind presence	2 of 6 OptiX build-input enum values; 2 of 4 leaf kinds
Bounded V4 corpus	4 of 6 build-input enum values; all 4 leaf kinds (triangle, custom, curve, sphere)
User-authored template	One bounded built-in-triangle template
Prospective composition	One sealed-snapshot sphere composition
Human usability	No independent-user authoring evidence or timing

5 Evaluation

We ask whether mutations reach intended gates, a sealed framework admits one selected composition, an independent checker validates generated artifacts, and the public route approaches direct performance at registered endpoints.

5.1 Earlier liveness and residual evidence

For two fixed contracts, 19 single-leaf mutations exercised 19/19 expected gates: seven role/effect, one semantic-ABI, seven physical-ABI, two continuation, and two identity mutations. This denominator covers only those contracts and mutations and is not pooled with later experiments.

Corrupting each target projection produced the expected sole reason before native-library load. For semantic ABI, this is mutation sensitivity, not an execution of the illustrative wrong-row route. Historical PyOptiX/OWL-style diagnostics lack independently verifiable execution records

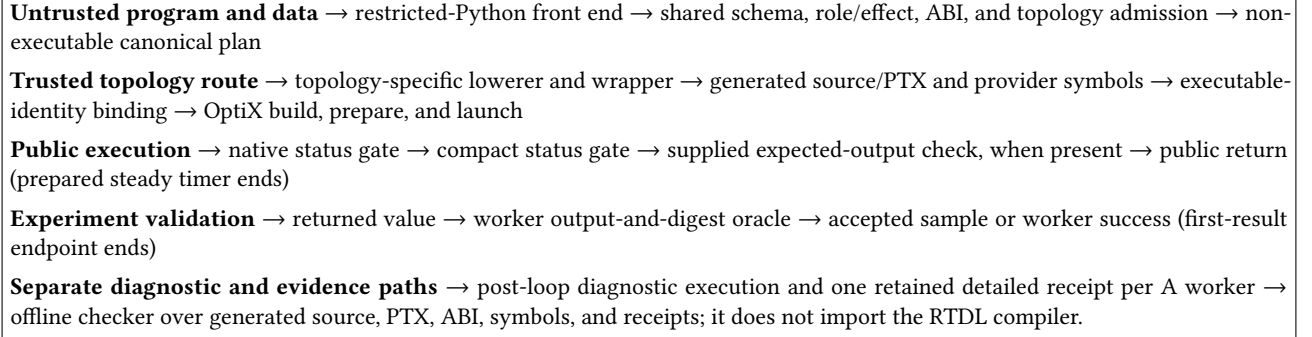


Figure 1. Architecture and trust boundary. Shared admission does not replace the topology-specific lowerer; that lowerer remains in the TCB.

here and are not evidence about those systems' guarantees [15, 16]. The 19/19 result is neither soundness nor prevalence evidence.

5.2 Sealed-snapshot composition exam

Before a public randomness pulse, we fixed an author-defined ten-row challenge: seven built-in four-role and three custom six-role rows. All used supported primitive families, and every continuation was a per-query count. The curve any-hit-terminate row remained eligible but unselected. The pulse selected builtin_sphere::any_hit_count_continue_u64_per_query.

Private custody identifies the sealed source. No bytes changed in three frozen schema, admission, and lifecycle files. The route made two OptiX launches and passed 12/12 rational-oracle rows, showing reuse of admission, identity, and lifecycle for this composition. It is not an unbiased application: authors defined the domain, rows recombined known ingredients, and the selected route required substantial topology-specific code.

5.3 Independent finite checker

The offline checker imports no RTDL module or compiler projection. It reads generated source, PTX/ABI metadata, symbols, and receipts. Across three route groups and four modes, it applied five property classes 20 times using 15 unique mutations, checking selected target-code patterns and ordering constraints.

This checker is finite, not a verifier for arbitrary device control flow. During review, an out-of-authority in-memory probe inserted an unconditional early return and still passed selected effect/status helper checks. The result therefore supports specialized target-structure validation only. General control-flow, numerical, and refinement soundness remain open; tools such as GPUVerify illustrate the substantially stronger proof obligation [1].

5.4 Performance methodology

We measured two exact tasks. *Relation* uses 4,096 objects and 4,096 queries, returns 4,096 canonical u32 pair rows, and performs two true OptiX traversals. *Triangle* uses 16,384 primitives and 16,384 queries, performs one true traversal, and returns the checked scalar 65,530.

Five arms isolate different boundaries:

- A current RTDL ahead-of-time public path at measured source M;
- B idiomatic pinned PyOptiX-compatible baseline;
- C strong PyOptiX-compatible baseline with device continuation;
- D named Direct CUDA/OptiX reference implementation;
- E frozen predecessor control for RTDL.

We ran the same registered matrix on an RTX 4090 (Ada) and RTX 3090 (Ampere). Each generation has eight blocks, two tasks, five arms, 80 cells, and 128 steady samples per cell: 160 cells and 20,480 samples total. Ratios are medians of eight within-block ratios of integer nanoseconds; lower is better, and raw times are not compared across machines. The steady timer encloses a prepared public action through return; validation follows. Entry starts before implementation-specific imports, while post-import starts after them; both end after first-result validation. PyOptiX initializes CUDA during import, whereas RTDL does so later, so these intervals include different lifecycle work.

The registered primary criterion was prepared A/D median ≤ 1.20 , with no maximum-block gate. A/C entry used median ≤ 1.20 and maximum ≤ 1.35 , but the endpoint was revised after an adverse result and both first-result endpoints remain confounded. We report A/C only diagnostically. Only prepared A/E was a registered regression gate; A/E first-result values are post hoc. C/B competence used ≤ 1.05 .

5.5 Prepared execution results

Table 5 reports the main observation. The public AOT path is 1.077–1.175× the direct implementation by median. The

Table 5. Prepared public RTDL versus direct CUDA/OptiX (A/D). Ratio is lower-is-better; times are medians of worker medians.

GPU	Task	A (ms)	D (ms)	Median / max
RTX 4090	Relation	0.2810	0.2612	1.0769 / 1.0923
RTX 4090	Triangle	0.0598	0.0510	1.1751 / 1.2110
RTX 3090	Relation	0.2402	0.2198	1.0948 / 1.1188
RTX 3090	Triangle	0.0608	0.0536	1.1336 / 1.1427

triangle maximum block on Ada is 1.211 $\times$; because no maximum A/D gate was registered, it is reported as dispersion rather than converted into a pass/fail criterion.

Across 20 observations, AOT cache-hit medians were 58.3–78.2 ms, about 1.04–2.02% of cold time. Hits still include lookup, validation, and load and are outside the prepared endpoint.

5.6 Adverse lifecycle results

Every A/C post-import median is adverse, 1.560–1.837 $\times$, with a 2.377 $\times$ worst block. Because entry was revised after the adverse result, neither endpoint supports a confirmatory claim. Against E, A also regresses by about 8–22% at entry and 16–31% post-import despite favorable prepared A/E, showing material pre-steady-state protocol and Python/native lifecycle work.

Arm C outperformed B in all four formal steady comparisons (C/B ratios 0.221–0.654), establishing that B alone would be a weak baseline for this experiment. It does not establish C as globally optimal. We also recorded 1,024 timer-instrumentation endpoints for A only. Paired overhead was measured only for A; no corresponding paired overhead measurement was made for B, C, D, or E.

No wrong output was observed in the final GPU samples. This statement is not a correctness proof and does not repair the detailed-receipt shortfall described in Section 3.6.

6 Programming Responsibility and Generality

Users provide bounded callback intent and data; shared RTDL machinery validates schemas, cross-role effects, ABI, identity, and lifecycle; trusted per-topology lowerers generate and bind CUDA/OptiX code. Failed status gates suppress output, while experiments may add application oracles.

The implementation suggests, but does not validate elsewhere, a pattern for managed-language/accelerator boundaries: expose cross-component obligations, compare source and target projections, gate executable minting, bind admission to loaded bytes, and include asynchronous failure in publication. Lowerers, trusted semantics, and checker vocabularies must be re-established per domain.

7 Related Work

OptiX established programmable RT [17]. Current OptiX, Vulkan, and DXR specify extensive local pipeline, interface, payload, and SBT rules [10, 11, 13, 14]. DXR payload access qualifiers express per-stage reads/writes and enforce declaration rules, although local-copy access still carries a documented developer responsibility. Vulkan assigns applications SBT organization under its indexing rules. These are substantial guarantees, not unchecked APIs.

PyOptiX provides Python bindings for the OptiX host API, while SlangPy includes Python RT pipeline, hit-group, shader-table, and dispatch facilities [9, 15]. OWL manages substantial OptiX construction, and Luisa automates resource dependency analysis and RT pipeline/SBT/launch work [12, 16, 22]. The checked primary materials do not state the same joint five-seam admission condition; this source-bounded observation is not evidence that those systems cannot implement it.

Shader Components and Slang provide modular interfaces, specialization, and reflection; current Slang also infers and validates target, stage, API, and hardware capabilities, while shader cursors address cross-platform parameter layout and binding [5, 6, 19, 20]. Dr.Jit traces and optimizes broad Python/C++ computations through OptiX, and CrossRT generates cross-platform host/device RT code [3, 8]. RTDL is narrower: it contributes a concrete route-level admission organization, not their language breadth, generation scope, or an exclusivity result.

Typestate, interface automata, and multiparty session types provide general and often formally stronger models of state and global/local protocol compatibility [2, 7, 21]. Typed linking and FFI checking already relate rich cross-language and physical representations [4, 18]. RTDL specializes these concerns to five RT compiler/runtime seams and proves no new calculus or soundness theorem. Its finite target checker is empirical and structural, unlike formal GPU verification such as GPUVerify [1].

8 Artifact

The evidence-only nine-member archive contains an anonymous projection, summary, manifest, documentation, inventory, and standard-library Python verifier. Private provenance and export receipts remain outside it. The projection records 160 formal cells, 20,480 steady samples, 1,024 A-only instrumentation endpoints, 20 AOT samples, and eight competence records.

Two clean exports were byte-identical. Replays in two fresh directories, one with spaces, passed in isolated normal and optimized Python modes. The archive bundles no third-party code, data, CUDA/OptiX material, driver, key, or raw GPU archive. It verifies submitted evidence offline, not GPU rerun, installation, or private-history reconstruction; new execution needs external CUDA/OptiX and GPU.

Table 6. Lifecycle ratios. A/C compares current RTDL with the strong device-continuation baseline; A/E compares current with frozen predecessor control E. A/E entry and post-import rows are post hoc, non-gating diagnostics; only A/E prepared was registered. Ratios are lower-is-better.

GPU	Task	A/C entry	A/C post median	A/C post range	A/E entry	A/E post	A/E prepared
RTX 4090	Relation	0.654	1.749	1.725–1.866	1.192	1.305	0.584
RTX 4090	Triangle	0.642	1.560	1.527–1.639	1.080	1.169	0.903
RTX 3090	Relation	0.681	1.837	1.816–2.377	1.217	1.262	0.608
RTX 3090	Triangle	0.618	1.637	1.608–1.653	1.138	1.163	0.922

9 Threats to Validity

Construct validity. Prepared A/D measures the public steady path after setup; it does not measure interactive authoring, cold compilation, or first use. Direct D is the named contract-equivalent reference implementation, not a proven minimum-latency implementation or an application framework. Strong C is contract-equivalent at the output boundary but is neither byte-identical nor operation-identical to A. These observations do not identify intrinsic language overhead. Paired instrumentation overhead was measured only for A, not for the other arms. The same-width semantic-ABI example is illustrative: mutation tests establish check liveness, but no retained run executes its defective route. A trusted semantic schema may also be wrong while internally coherent.

Internal validity. Both tasks were used during adaptation and are not unseen workloads. Entry was revised after an adverse result, and both first-result endpoints include import and lifecycle effects; neither is reinterpreted as a pass. Mocks reproduce the provider double fault, which can replace the primary exception and lose cleanup retry ownership. Native forks bypassing the at-fork hook can evade the cached-PID guard. Neither unrepaired path produced output or appeared in retained GPU work.

External validity. Measurements cover two NVIDIA generations and two exact tasks; raw latency is not compared across machines. Authors defined the challenge domain from known ingredients and selected one composition. There is no independent-user authoring evidence, prevalence measure, or arbitrary-Python/IR evidence. Several related-work guarantee questions remain unknown; no first, only, or impossibility claim follows.

Implementation trust. The selected sphere route required about 2,635 lines of topology-specific TCB code. The finite checker missed an out-of-authority early-return probe for selected helpers. The 4,096 timed A executions lack per-execution receipts.

Artifact scope. The anonymous projection is derived evidence. Its verifier checks identities, counts, formulas, and disclosed rows, but cannot recreate GPU runs or custody; private provenance remains available for confidential audit.

10 Conclusion

RTDL admits bounded RT routes as protocol-carrying executables. One decision compares five cross-artifact seams; typed effects reach hardware control only through trusted topology wrappers. Two public constructors and a four-leaf corpus exercise the design. Finite mutation, composition, and target checks support this boundary, not semantic inference, arbitrary lowering, novelty by absence, or general soundness.

Across two tasks and GPU generations, prepared median latency is $1.077\text{--}1.175\times$ Direct; post-import A/C reaches $1.837\times$ median and a $2.377\times$ worst block. This bounded compiler-system method has near-direct prepared latency and explicit TCB, receipt, generality, and lifecycle limits.

References

- [1] Adam Betts, Nathan Chong, Alastair F. Donaldson, Shaz Qadeer, and Paul Thomson. 2012. GPUVerify: A Verifier for GPU Kernels. In *Proceedings of the ACM International Conference on Object-Oriented Programming Systems Languages and Applications*. ACM, New York, NY, USA, 113–132. <https://doi.org/10.1145/2384616.2384625>
- [2] Luca de Alfaro and Thomas A. Henzinger. 2001. Interface Automata. In *Proceedings of the 8th European Software Engineering Conference Held Jointly with the 9th ACM SIGSOFT International Symposium on Foundations of Software Engineering*. ACM, New York, NY, USA, 109–120. <https://doi.org/10.1145/503209.503226>
- [3] Vladimir Frolov, Vadim Sanzharov, Albert Garifullin, Maxim Raenchuk, and Alexei Voloboy. 2024. CrossRT: A Cross Platform Programming Technology for Hardware-Accelerated Ray Tracing in CG and CV Applications. arXiv:2409.12617. <https://doi.org/10.48550/arXiv.2409.12617>
- [4] Michael Furr and Jeffrey S. Foster. 2005. Checking Type Safety of Foreign Function Calls. In *Proceedings of the 2005 ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM, New York, NY, USA, 62–72. <https://doi.org/10.1145/1065010.1065019>
- [5] Yong He, Kayvon Fatahalian, and Theresa Foley. 2018. Slang: Language Mechanisms for Extensible Real-Time Shading Systems. *ACM Transactions on Graphics* 37, 4, Article 141 (2018), 13 pages. <https://doi.org/10.1145/3197517.3201380>
- [6] Yong He, Theresa Foley, Teguh Hofstee, Haomin Long, and Kayvon Fatahalian. 2017. Shader Components: Modular and High Performance Shader Development. *ACM Transactions on Graphics* 36, 4, Article 100 (2017), 11 pages. <https://doi.org/10.1145/3072959.3073648>
- [7] Kohei Honda, Nobuko Yoshida, and Marco Carbone. 2016. Multiparty Asynchronous Session Types. *J. ACM* 63, 1, Article 9 (2016), 67 pages. <https://doi.org/10.1145/2827695>
- [8] Wenzel Jakob, Sébastien Speierer, Nicolas Roussel, and Delio Vicini. 2022. DrJit: A Just-In-Time Compiler for Differentiable Rendering.

- ACM Transactions on Graphics 41, 4 (2022), 1–19. <https://doi.org/10.1145/3528223.3530099>
- [9] Simon Kallweit, Chris Cummings, Benedikt Bitterli, Sai Bangaru, and Yong He. 2026. SlangPy. <https://slangpy.shader-slang.org/en/stable/>. Stable documentation, version 0.42.0.
- [10] Khronos Group. 2026. Vulkan Specification: Ray Tracing and Shader Binding Tables. <https://docs.vulkan.org/spec/latest/chapters/raytracing.html>.
- [11] Khronos Group. 2026. Vulkan Specification: Shader Interfaces. <https://docs.vulkan.org/spec/latest/chapters/interfaces.html>.
- [12] LuisaGroup. 2026. LuisaCompute: High-Performance Rendering Framework on Stream Architectures. <https://github.com/LuisaGroup/LuisaCompute>. Official repository and current Python-frontend documentation.
- [13] Microsoft. 2026. DirectX Raytracing Functional Specification: Payload Access Qualifiers. <https://microsoft.github.io/DirectX-Specs/d3d/Raytracing.html>.
- [14] NVIDIA. 2025. *NVIDIA OptiX 9.0 Programming Guide*. NVIDIA. https://raytracing-docs.nvidia.com/optix9/guide/optix_guide.250130.A4.pdf Payload types and per-stage payload semantics.
- [15] NVIDIA. 2026. PyOptiX: Complete Python Bindings for the OptiX Host API. <https://github.com/NVIDIA/otk-pyoptix>. Pinned experiment source commit 3144f224c0fd18733925faf3d8fb82c7376b8dcf.
- [16] NVIDIA, Ingo Wald, Stefan Zellmann, and contributors. 2026. OWL: The OptiX Wrappers Library. <https://github.com/NVIDIA/OWL>. Experiment pins upstream commit df7390b16bce5244b7352ca6d3e320f838297072.
- [17] Steven G. Parker, James Bigler, Andreas Dietrich, Heiko Friedrich, Jared Hoberock, David Luebke, David McAllister, Morgan McGuire, Keith Morley, Austin Robison, and Martin Stich. 2010. OptiX: A General Purpose Ray Tracing Engine. *ACM Transactions on Graphics* 29, 4, Article 66 (2010), 13 pages. <https://doi.org/10.1145/1778765.1778803>
- [18] Daniel Patterson and Amal Ahmed. 2017. Linking Types for Multi-Language Software: Have Your Cake and Eat It Too. In *2nd Summit on Advances in Programming Languages (Leibniz International Proceedings in Informatics, Vol. 71)*. Schloss Dagstuhl–Leibniz-Zentrum fuer Informatik, Dagstuhl, Germany, 12:1–12:15. <https://doi.org/10.4230/LIPIcs.SNAPL.2017.12>
- [19] Slang Project. 2026. Capabilities—Slang User’s Guide. <https://docs.shader-slang.org/en/stable/external/slang/docs/user-guide/05-capabilities.html>.
- [20] Slang Project. 2026. A Practical and Scalable Approach to Cross-Platform Shader Parameter Passing Using Slang’s Reflection API. <https://docs.shader-slang.org/en/stable/shader-cursors.html>.
- [21] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. <https://doi.org/10.1109/TSE.1986.6312929>
- [22] Shaokun Zheng, Zhiqian Zhou, Xin Chen, Difei Yan, Chuyan Zhang, Yuefeng Geng, Yan Gu, and Kun Xu. 2022. LuisaRender: A High-Performance Rendering Framework with Layered and Unified Interfaces on Stream Architectures. *ACM Transactions on Graphics* 41, 6, Article 232 (2022), 19 pages. <https://doi.org/10.1145/3550454.3555463>